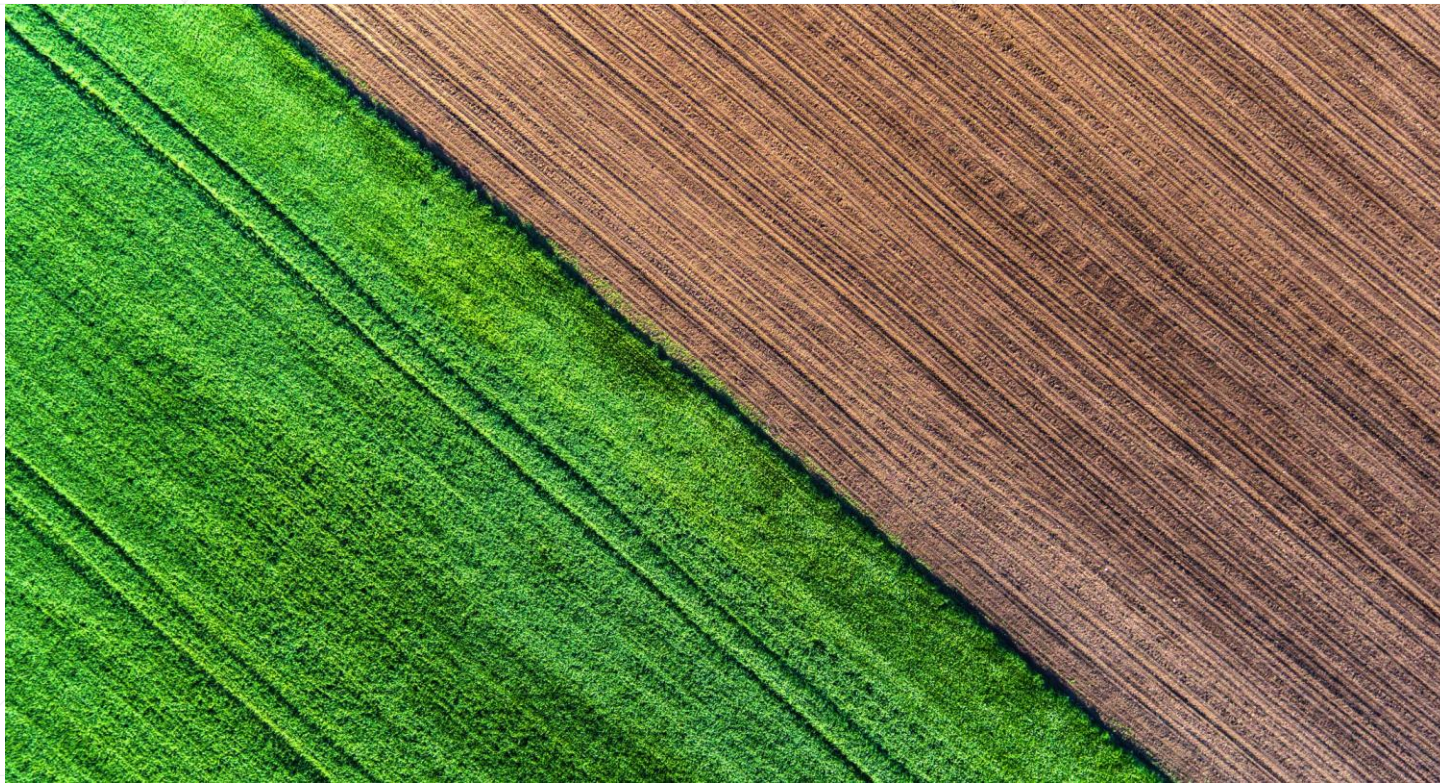




初识进程与线程（上）

北京理工大学计算机学院
金旭亮

进程



任务管理器				
文件(F) 选项(O) 查看(V)				
进程 性能 应用历史记录 启动 用户 详细信息 服务				
名称	8% CPU	26% 内存	1% 磁盘	37% 网络
应用 (4)				
> HyperSnap	0%	8.9 MB	0 MB/秒	0 Mbps
> Microsoft PowerPoint (32 位)	0.2%	67.9 MB	0 MB/秒	0 Mbps
> Task Manager	0.8%	13.3 MB	0 MB/秒	0 Mbps
> Windows 资源管理器	0%	37.1 MB	0 MB/秒	0 Mbps
后台进程 (35)				
> [Icon]	0%	2.6 MB	0 MB/秒	0 Mbps
> [Icon]	0%	1.7 MB	0 MB/秒	0 Mbps
> BaiduNetdisk (32 位)	1.4%	28.5 MB	0 MB/秒	0 Mbps
> Bonjour Service	0%	1.7 MB	0 MB/秒	0 Mbps
> Boot Camp Manager	0%	2.0 MB	0 MB/秒	0 Mbps
> Cortana (小娜)	0%	33.9 MB	0 MB/秒	0 Mbps
> Data Transfer Accelerator (32 位)	0%	2.4 MB	0 MB/秒	0 Mbps
> Device Association Framework Provider Host	0%	5.8 MB	0 MB/秒	0 Mbps
简略信息(D) 结束任务(E)				

每个正在运行的程序，计算机操作系统都将其视为一个“**进程 (Process)**”。

进程是操作系统分配各种资源（比如内存）的基本单位。



试验

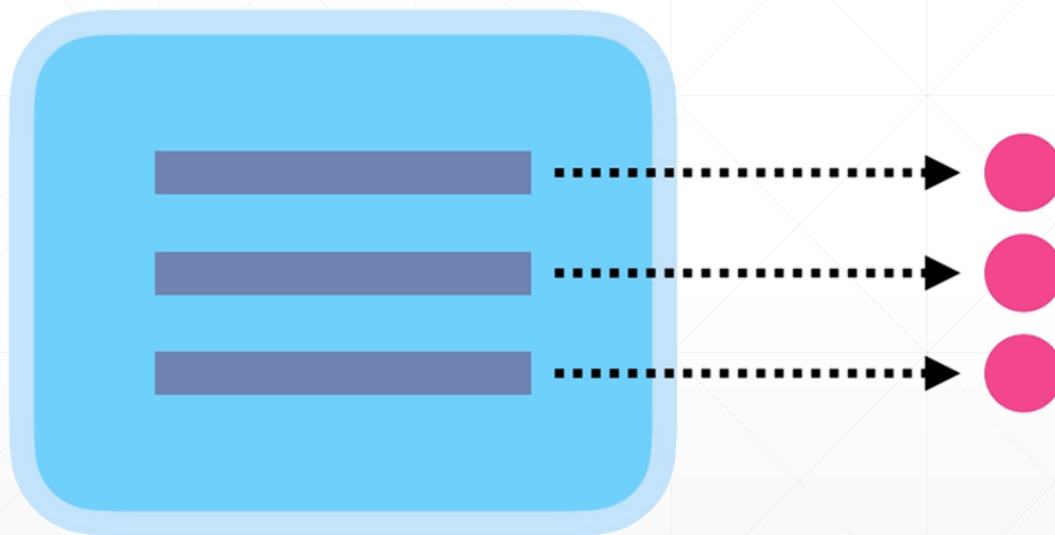
请编写一个简单的Java应用，使用java命令运行之，然后，打开任务管理器观察任务列表，看看是不是多了一个java进程.....

每个运行的Java程序，都是一个进程，对应着一个正在运行的“Java虚拟机（JVM）”进程。

任务管理器					
文件(F) 选项(O) 查看(V)					
进程 性能 应用历史记录 启动 用户 详细信息 服务					
名称	状态	5% CPU	19% 内存	0% 磁盘	1% 网络
应用 (6)					
> 1 进程基础.pptx - PowerPoint		0%	88.0 MB	0 MB/秒	0 Mbps
> HyperSnap		0%	23.9 MB	0.1 MB/秒	0 Mbps
> Windows 资源管理器 (2)		0.3%	169.1 MB	0 MB/秒	0 Mbps
> XMind (7)		0%	313.8 MB	0 MB/秒	0 Mbps
> 任务管理器		0.4%	34.7 MB	0 MB/秒	0 Mbps
▼ 终端 (5)		0%	71.3 MB	0 MB/秒	0 Mbps
Java(TM) Platform SE binary		0%	14.1 MB	0 MB/秒	0 Mbps
OpenConsole.exe		0%	1.6 MB	0 MB/秒	0 Mbps
Runtime Broker		0%	1.3 MB	0 MB/秒	0 Mbps
Windows PowerShell		0%	25.5 MB	0 MB/秒	0 Mbps
Windows PowerShell		0%	28.7 MB	0 MB/秒	0 Mbps
^ 简略信息(D) 结束任务(E)					



进程在运行过程中，可以创建多个“线程”。

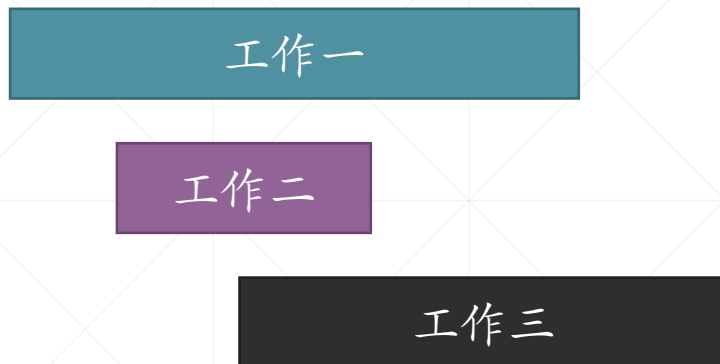


不同的线程，可以同时干不同的或相同的工作。

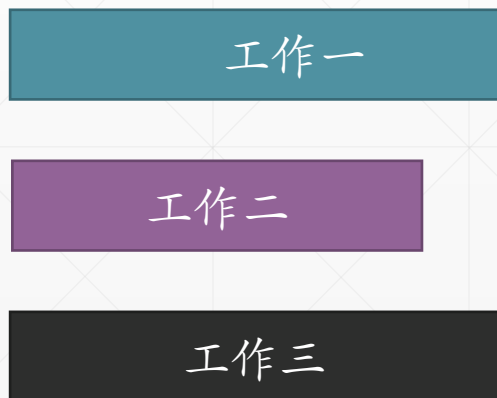
顺序
(Sequential)



并发
(Concurrent)



并行
(Parallel)





在开发多线程程序时，通常不预先假定这个程序会在拥有多少个核的CPU上并行运行，而是按照“并发”这种运行模式来设计和编写这个多线程程序。

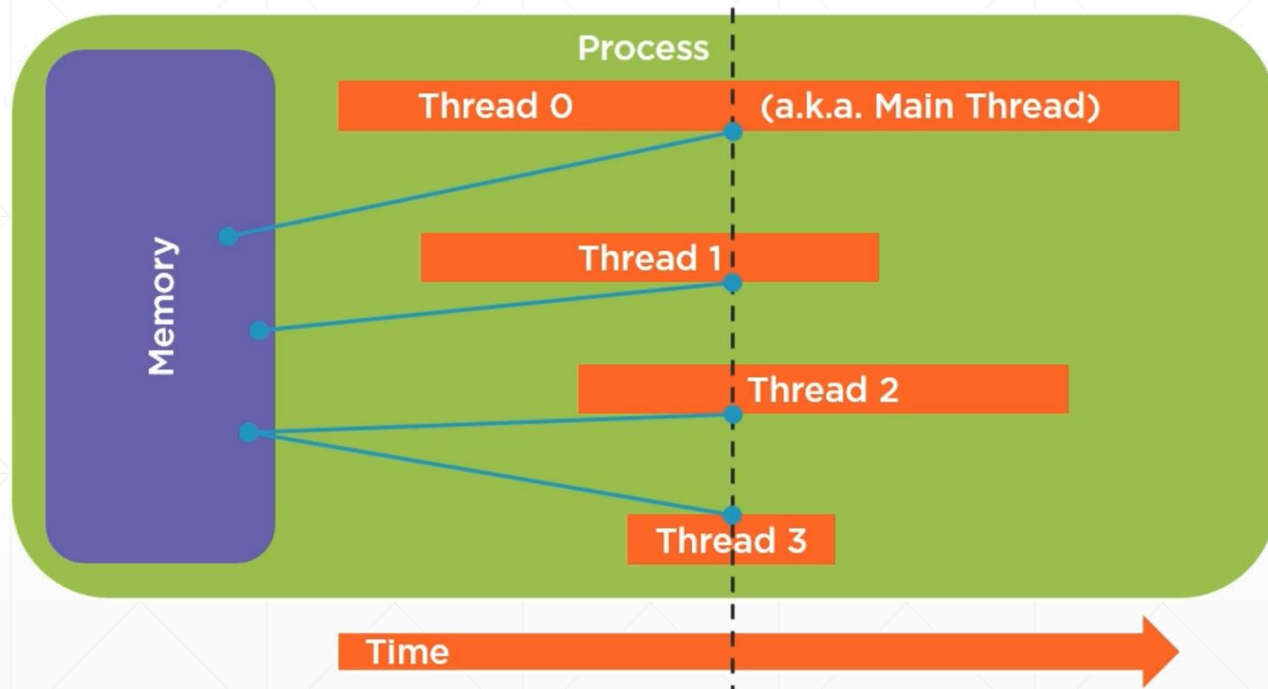


考虑了“并发”的多线程程序，通常情况下，在多个CPU执行核上“并行”运行，并不会引发数据存取错误。



如果确实需要“显式”地指定并行模式，则往往需要编写特定的代码，在程序运行时检测当前计算机的可用核数，然后，将数据进行分区，或者将数据处理过程进行“分段”，从而实现并行计算。这种“高度定制化”的程序，设计和编写都比较困难，通常在实际开发中，人们会使用专门设计的并行计算库或框架，而不是“一切自己动手从零开始”。

并发执行的线程，共享进程拥有的内存

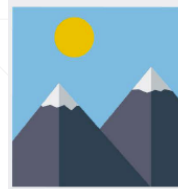


由于并发执行的线程，可以同时存取同一块内存区域，因此，如果事先不做好约定，就有可能由于读写顺序问题造成数据存取错误。



如何协调好多个线程访问共享资源的顺序，如何让线程之间可以相互协作，这部分内容，是“线程同步”需要关心的问题。

进程 vs. 线程



进程是操作系统分配内存及其他系统资源的基本单元，线程是操作系统分配CPU的基本单元，一个进程，可以创建多个线程。



不同进程的內部数据和状态是完全独立的，而多个线程则可以共享它所属的进程所拥有的资源，因此，线程之间会相互影响。



线程本身运行所需的环境比较简单，通常只包括寄存器及堆栈，切换速度比进程要快得多。



在开发中应用多线程技术的主要目的是“**最大限度地利用CPU，避免其长期居于闲置状态**”。比如，某一线程正在等待IO操作的结果才能继续，此时不需要占用CPU，就应该将CPU让给其他线程使用。



了解Java虚拟机进程

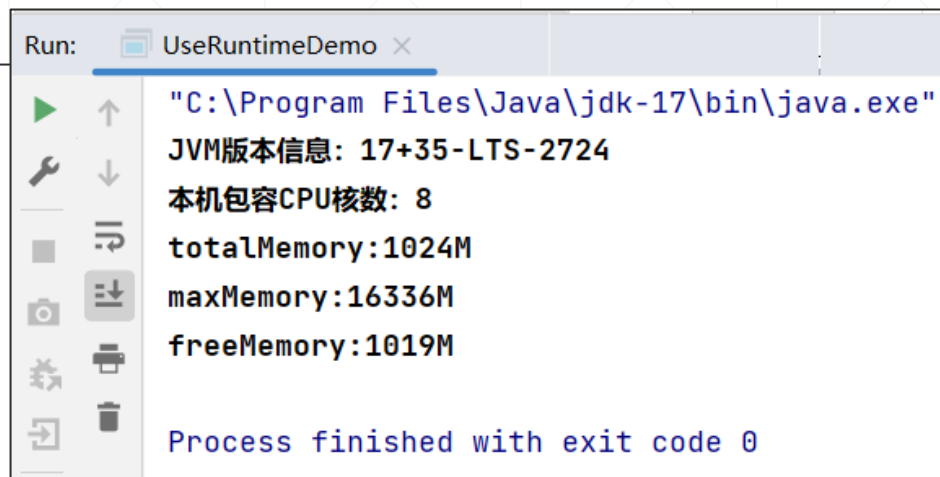
每个正在运行的Java程序，都对应着一个“Java虚拟机（JVM）进程”。

JDK中提供了一个Runtime类，提供了一些有用的方法，可以获取当前正在运行的Java虚拟机中许多有价值的信息。

Runtime		
m	exec(String, String[], File)	Process
m	exec(String[], String[], File)	Process
m	freeMemory()	long
m	halt(int)	void
m	removeShutdownHook(Thread)	boolean
m	maxMemory()	long
m	load(String)	void
m	totalMemory()	long
m	exit(int)	void
m	version()	Version
m	getRuntime()	Runtime
m	exec(String[], String[])	Process
m	exec(String, String[])	Process
m	addShutdownHook(Thread)	void
m	exec(String)	Process
m	loadLibrary(String)	void
m	exec(String[])	Process
m	gc()	void
m	availableProcessors()	int
m	runFinalization()	void

使用Runtime获取系统信息

```
public class UseRuntimeDemo {  
    public static void main(String[] args) {  
        System.out.println("JVM版本信息: " + Runtime.version());  
        //获取当前虚拟机运行时引用  
        var runtime : Runtime = Runtime.getRuntime();  
        int CPU = runtime.availableProcessors();  
        System.out.println("本机包容CPU核数: " + CPU);  
  
        //当前虚拟机内存总量  
        long totalMemory = runtime.totalMemory();  
        System.out.println("totalMemory:" + totalMemory / (1024 * 1024) + "M");  
        //当前虚拟机可用最大内存数  
        long maxMemory = runtime.maxMemory();  
        System.out.println("maxMemory:" + maxMemory / (1024 * 1024) + "M");  
        //当前虚拟机可用内存数  
        long freeMemory = runtime.freeMemory();  
        System.out.println("freeMemory:" + freeMemory / (1024 * 1024) + "M");  
    }  
}
```



Run: UseRuntimeDemo x

"C:\Program Files\Java\jdk-17\bin\java.exe"

JVM版本信息: 17+35-LTS-2724

本机包容CPU核数: 8

totalMemory:1024M

maxMemory:16336M

freeMemory:1019M

Process finished with exit code 0

当Java虚拟机关闭时.....

如果想在JVM关闭时完成特定的工作，可以使用Hook线程。

Runtime类定义了以下方法，支持挂接多个Hook线程，当JVM关闭时会回调这些线程：

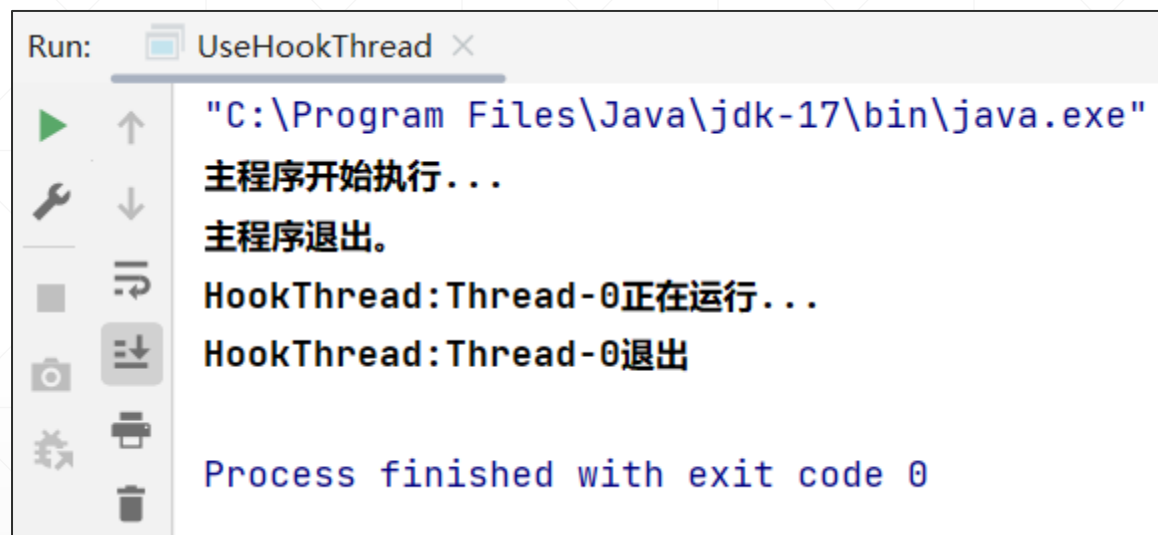
```
public class Runtime {  
  
    public void addShutdownHook(Thread hook) {  
    }  
  
    public boolean removeShutdownHook(Thread hook) {  
    }  
  
}
```

```
public class UseHookThread {  
    public static void main(String[] args) {  
        Runnable runnable = () -> {  
            var threadName = "HookThread:" + Thread.currentThread().getName();  
            System.out.println(threadName + "正在运行...");  
            try {  
                TimeUnit.SECONDS.sleep(timeout: 1);  
            } catch (InterruptedException e) {  
                throw new RuntimeException(e);  
            }  
            System.out.println(threadName + "退出");  
        };  
        System.out.println("主程序开始执行...");  
        Runtime.getRuntime().addShutdownHook(new Thread(runnable));  
        System.out.println("主程序退出。");  
    }  
}
```

使用Runnable
封装要干的活

UseHookThread.java

运行截图



关于Hook线程



Hook线程会在JVM退出时运行。



Hook线程中可以执行一些资源释放的工作，比如关闭文件句柄、socket、数据库连接等。



尽量不要在Hook线程中执行一些耗时非常长的操作，因为这会导致JVM迟迟不能退出。